

LangChain / LangGraph Agent Architecture

Agent Types	Graph Layers	Scaling Strategy	Human Control
6	3	Horizontal + Hierarchical	Full Veto Power

Table of Contents

01	Overview & Architecture Philosophy
02	Agent Types Deep Dive
2	.
1	Material Flow Intelligence Agent
2	.
2	Local Negotiation Agent
2	.
3	Super / Locality Coordinator Agent
2	.
4	Specialist Agents (Recycler, Logistics, Energy)
2	.
5	Digital Passport Agent
2	.
6	Orchestrator / Planner Agent
03	LangGraph State Machine Design
04	LangChain Integration Patterns
05	Agent Communication Protocol
06	Scaling Strategy
07	Human-in-the-Loop Control Layer
08	Implementation Roadmap

01 Overview & Architecture Philosophy

Why LangChain + LangGraph for EcoNexus

EcoNexus v2.0 replaces its rule-based coordination engine with a fully graph-driven, LLM-augmented multi-agent system. LangGraph provides the **stateful, cyclical execution graphs** needed for multi-round negotiations, while LangChain's tool ecosystem handles external integrations—databases, APIs, embeddings—without reinventing the wheel.

Why LangGraph over vanilla LangChain Agents?

Capability	LangChain AgentExecutor	LangGraph StateGraph
Cyclic execution	■ No native cycles	■ First-class support
State persistence	■ Per-turn only	■ Typed state across nodes
Multi-agent comms	■ Manual wiring	■ Graph edges + channels
Parallel branches	Limited	■ Parallel node execution
Human checkpoints	Partial	■ <code>interrupt_before</code> / <code>interrupt_after</code>
Sub-graph nesting	■ Not supported	■ Composable sub-graphs
Streaming	■ Supported	■ Supported + node-level

Three-Tier Architecture

The system is organized into three concentric rings of agency, each with escalating scope and authority:

Tier	Layer	Agent Type	Scope	Decision Power
1	Micro	Local Agent (per company)	50 km radius	Propose & Negotiate
2	Meso	Super Agent (per locality)	Industrial park	Structure Multi-Party
3	Macro	Specialist Agents	National / regional	Market-Making

■ Core Design Principle

Agents never finalize deals. Every negotiated outcome is a *proposal* routed to a human approver via LangGraph's interrupt mechanism. This keeps AI in advisory mode while eliminating the coordination drudgery.

02 Agent Types — Deep Dive

Six specialized agents, each a LangGraph StateGraph

2.1 Material Flow Intelligence (MFI) Agent

The MFI Agent is the **data backbone** of EcoNexus. It continuously ingests invoices, ERP exports, and sensor streams, classifying materials, computing KPIs, and publishing structured waste-stream records to the shared state store.

State Schema

mfi_state.py

```
# langgraph typed state for MFI Agent
from typing import TypedDict, List, Optional
from langchain_core.messages import BaseMessage

class MFISState(TypedDict):
    raw_invoices: List[dict] # uploaded invoice payloads
    parsed_materials: List[dict] # extracted line items
    waste_streams: List[dict] # classified waste records
    kpis: dict # MCI, carbon, diversion rate ...
    confidence_scores: List[float] # per-item extraction confidence
    errors: List[str] # validation failures
    messages: List[BaseMessage]
```

Graph Nodes

Node	LangChain Component	Responsibility
ingest_node	DocumentLoader + PDFParser tool	Load & chunk invoice PDFs
extract_node	LLM Chain (Claude / GPT-4o)	NLP extraction of materials & quantities
classify_node	Few-shot classifier chain	Map materials to EWC waste codes
normalize_node	PydanticOutputParser	Unit normalization, dedup
kpi_node	Python tool (rule-based)	Compute MCI, carbon, diversion
publish_node	Supabase tool / API call	Persist to DB, emit events
validation_node	Structured output chain	Confidence gating & human escalation

mfi_graph.py

```
from langgraph.graph import StateGraph, END
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model='claude-sonnet-4-20250514')

def extract_node(state: MFISState) -> MFISState:
    """Use LLM to extract material line items from raw invoice text."""
    chain = extraction_prompt | llm | PydanticOutputParser(pydantic_object=MaterialList)
```

```

result = chain.invoke({'invoices': state['raw_invoices']})
return {'**state', 'parsed_materials': result.items}

def route_confidence(state: MFISState) -> str:
    """Conditional edge: low-confidence items go to human review."""
    avg = sum(state['confidence_scores']) / len(state['confidence_scores'])
    return 'publish_node' if avg > 0.85 else 'validation_node'

mfi_graph = StateGraph(MFISState)
mfi_graph.add_node('ingest', ingest_node)
mfi_graph.add_node('extract', extract_node)
mfi_graph.add_node('classify', classify_node)
mfi_graph.add_node('normalize', normalize_node)
mfi_graph.add_node('kpi', kpi_node)
mfi_graph.add_node('publish', publish_node)
mfi_graph.add_node('validation', validation_node)
mfi_graph.add_conditional_edges('normalize', route_confidence)
mfi_graph.set_entry_point('ingest')
mfi_agent = mfi_graph.compile()

```

2.2 Local Negotiation Agent

One instance per company. This agent owns the full lifecycle from **opportunity discovery** → **autonomous negotiation** → **deal packaging** → **human proposal**. It uses an LLM for complex counter-offer generation while falling back to rule-based logic for 90% of routine price / volume matches.

local_agent_state.py

```
class LocalAgentState(TypedDict):
    company_id: str
    waste_streams: List[dict] # from MFI agent
    constraints: dict # price_min/max, max_distance, materials
    feed_posts: List[dict] # current agent feed snapshot
    opportunities: List[dict] # scored matches
    active_negotiation: Optional[dict] # current thread
    negotiation_rounds: int
    deal_proposal: Optional[dict] # ready for human
    messages: List[BaseMessage]
    human_decision: Optional[str] # "approve" | "reject" | "edit"
```

Negotiation Loop — Cyclic Graph

The negotiation node forms a **cycle** — the defining capability of LangGraph that makes it ideal here. The agent iterates rounds until agreement, timeout, or a human interrupt is triggered.

negotiation_loop.py

```
def negotiate_node(state: LocalAgentState) -> LocalAgentState:
    """LLM-powered counter-offer or accept decision."""
    if state['negotiation_rounds'] >= 8:
        return {**state, 'active_negotiation': {**state['active_negotiation'],
        'status': 'timeout'}}
    chain = negotiation_prompt | llm.with_structured_output(NegotiationMove)
    move = chain.invoke({
        'offer': state['active_negotiation'],
        'constraints': state['constraints'],
        'rounds': state['negotiation_rounds']
    })
    return {**state,
        'active_negotiation': move.dict(),
        'negotiation_rounds': state['negotiation_rounds'] + 1}

def route_negotiation(state: LocalAgentState) -> str:
    neg = state['active_negotiation']
    if neg['status'] == 'accepted': return 'package_deal_node'
    if neg['status'] == 'timeout': return 'scan_opportunities_node'
    return 'negotiate_node' # loop back

# Human interrupt before deal is sent
local_graph = local_graph.compile()
```

```
interrupt_before=['send_proposal_node']  
)
```

■ Performance Note

Rule-based scoring (price match, distance filter) runs in **<5 ms** per opportunity. The LLM is invoked only when counter-offer language needs to be generated or deal terms are ambiguous — keeping API costs near zero for standard flows.

2.3 Super / Locality Coordinator Agent

One instance per industrial park. The Super Agent uses **graph algorithms + LLM reasoning** to detect circular loops ($A \rightarrow B \rightarrow C \rightarrow A$), structure multi-party deals, and negotiate inter-locality bulk trades with peer Super Agents.

super_agent_state.py

```
class SuperAgentState(TypedDict):
    locality_id: str
    member_agents: List[str] # company agent IDs
    material_balance: dict # surplus / deficit by type
    symbiosis_candidates: List[dict] # detected loop opportunities
    active_multiparty: Optional[dict] # ongoing 3-way+ deal
    cross_locality_queue: List[dict] # proposals to/from peers
    locality_kpis: dict # aggregate carbon, diversion
    messages: List[BaseMessage]
```

Symbiosis Detection Sub-Graph

symbiosis_detection.py

```
# Uses networkx for graph cycle detection, LLM for value calculation
import networkx as nx

def build_flow_graph_node(state: SuperAgentState) -> SuperAgentState:
    """Build directed graph: node=company, edge=material_flow."""
    G = nx.DiGraph()
    for agent_id in state['member_agents']:
        flows = fetch_waste_streams(agent_id)
        for flow in flows:
            G.add_edge(flow['producer'], flow['consumer'],
                material=flow['type'], volume=flow['volume'])
    cycles = list(nx.simple_cycles(G))
    return {**state, 'symbiosis_candidates':
        [score_cycle(G, c) for c in cycles if len(c) >= 2]}

def structure_deal_node(state: SuperAgentState) -> SuperAgentState:
    """LLM structures the best cycle into a multi-party deal."""
    best = max(state['symbiosis_candidates'], key=lambda c: c['value'])
    chain = deal_structuring_prompt | llm.with_structured_output(MultiPartyDeal)
    deal = chain.invoke({'cycle': best, 'balance': state['material_balance']})
    return {**state, 'active_multiparty': deal.dict()}
```

2.4 Specialist Agents

Specialist Agents act as **market-makers** — always-on buyers/sellers that provide price floors, logistics optimization, and last-resort disposal. They listen to the entire agent feed and respond autonomously.



Recycler Agent

Universal buyer · National scope

- Subscribes to all OFFER posts across localities
- Posts baseline buy-prices as price-floor signals
- Competes with local buyers driving price discovery
- Stateless: no negotiation rounds needed



Logistics Agent

Transport optimizer · National scope

- Route consolidation across deals
- Shared truck scheduling (TSP optimization)
- Cost estimate tool used by all negotiating agents
- Reduces distance-cost barrier for cross-locality deals



Processor Agent

Value transformer · Regional scope

- Buys low-value waste, sells processed materials
- Arbitrage: identifies transformation opportunities
- Enables materials that require pre-processing
- Posts both REQUEST and OFFER to feed



Energy Recovery Agent

Last-resort buyer · Regional scope

- Accepts non-recyclable / mixed materials
- Guarantees zero-landfill option is always available
- Posts standing buy-price for combustion-grade waste
- Updates prices based on energy market rates

Specialist Agent Implementation Pattern

All specialist agents share a **ReactAgent** base using LangChain's `create_react_agent` with a small tool set (`feed_search`, `post_offer`, `price_lookup`). Their graphs are single-cycle: `scan` → `evaluate` → `respond` → `sleep`.

2.5 Digital Passport Agent

Creates and validates material passports — immutable records that travel with each batch from producer to consumer. Uses LLM-based document verification and issues quality-tier scores (1–4) that other agents use in scoring.

Node	Action
create_passport	Generate UUID, capture origin, composition, carbon data
verify_documents	LLM checks uploaded certs against passport claims
issue_qr	Generate QR linking to passport record in DB
update_custody	Append transfer event when deal is approved
flag_anomaly	Alert if composition changes unexpectedly between transfers

2.6 Orchestrator / Planner Agent

The **top-level coordinator** that decides which agents to invoke, manages agent lifecycles, and handles cross-cutting concerns like network health, conflict resolution between competing proposals, and reporting.

```
orchestrator.py

# Orchestrator uses LangGraph's Send API for fan-out
from langgraph.constants import Send

def orchestrator_node(state: OrchestratorState):
    """Fan out to all active local agents in parallel."""
    return [Send('local_agent_graph', {'company_id': cid,
    'feed_snapshot': state['feed']})
    for cid in state['active_company_ids']]

# Collect results back into orchestrator state
def aggregate_results(state: OrchestratorState) -> OrchestratorState:
    proposals = [r for r in state['agent_results'] if r['type']=='proposal']
    conflicts = detect_conflicts(proposals) # same material, competing bids
    resolved = resolve_by_score(conflicts)
    return {**state, 'pending_proposals': resolved}
```

03 LangGraph State Machine Design

Full negotiation lifecycle as a typed StateGraph

The complete deal lifecycle — from waste stream detection to human approval — maps onto a single **top-level StateGraph** with sub-graphs for each agent type. LangGraph's persistence layer (checkpointer) ensures every state transition is durable and resumable.

Negotiation State Flow

Stage	Graph Node	Condition to Advance
1. Detect	confi_agent (sub-graph)	Waste stream confidence > 0.85
2. Broadcast	local_agent:broadcast_node	Post published to agent feed
3. Match	local_agent:scan_node	Opportunity score > threshold
4. Negotiate	local_agent:negotiate_node *	Status = 'accepted' OR rounds >= 8
5. Package	local_agent:package_deal_node	All terms structured
6. Interrupt	HUMAN CHECKPOINT	Human inputs approve/reject/edit
7. Execute	local_agent:execute_node	Both parties approved
8. Pass	passport_agent (sub-graph)	Custody record created

* Cyclic node — executes multiple times per deal

Shared State & Channels

```
global_state.py

from langgraph.graph import add_messages
from typing import Annotated

class EcoNexusGlobalState(TypedDict):
    # Append-only message channel (all agent comms)
    messages: Annotated[list, add_messages]

    # Agent feed – append-only list of posts
    agent_feed: Annotated[list, lambda x, y: x + y]

    # Deal proposals awaiting human action
    pending_proposals: list

    # Human decisions keyed by proposal_id
    human_decisions: dict

    # Active deals (approved, in execution)
    active_deals: list
```

Persistence & Checkpointing

LangGraph's SqliteSaver (dev) or PostgresSaver (prod) persists every state snapshot. This means a negotiation can be paused overnight while awaiting human approval and resumed exactly where it left off — even across server restarts.

persistence.py

```
from langgraph.checkpoint.postgres import PostgresSaver

checkpointer = PostgresSaver.from_conn_string(
    conn_string=os.environ["SUPABASE_DB_URL"]
)

# Compile with checkpointing + human interrupt
compiled_agent = local_graph.compile(
    checkpointer=checkpointer,
    interrupt_before=['send_proposal_node'],
)

# Resume after human decision
compiled_agent.invoke(
    {'human_decision': 'approve'},
    config={'configurable': {'thread_id': proposal_id}}
)
```

04 LangChain Integration Patterns

Tools, chains, and memory for each agent

Tool Registry

Each agent is given only the tools it needs — following least-privilege principles and keeping token usage low.

Tool Name	Used By	Purpose
feed_search_tool	Local, Specialist	Semantic search on agent feed posts
post_to_feed_tool	All agents	Create OFFER / REQUEST / NEGOTIATION post
price_lookup_tool	Local, Super	Fetch market price for material type
geo_distance_tool	Local	Haversine distance between two postcodes
route_cost_tool	Logistics Agent	Estimate transport cost (distance + load)
passport_fetch_tool	Local, Passport Agent	Retrieve or create digital passport record
supabase_upsert_tool	MFI, Passport Agent	Write structured data to Supabase
carbon_calc_tool	MFI, Super Agent	Compute CO ₂ saved per material exchange
notify_human_tool	Local, Super Agent	Push notification for deal approval

Semantic Search on Agent Feed

feed_search.py

```
from langchain_community.vectorstores import SupabaseVectorStore
from langchain_anthropic import AnthropicEmbeddings

embeddings = AnthropicEmbeddings()
vector_store = SupabaseVectorStore(
    client=supabase_client, embedding=embeddings,
    table_name="agent_feed_embeddings", query_name="match_posts"
)

@tool
def feed_search_tool(query: str, k: int = 5) -> List[dict]:
    """Semantically search agent feed for matching offers/requests."""
    docs = vector_store.similarity_search_with_score(query, k=k)
    return [{'post': d.page_content, 'score': s, 'meta': d.metadata}
            for d, s in docs]
```

Agent Memory Patterns

Memory Type	LangChain Component	Used For
Short-term (conversation)	InMemoryChatMessageHistory	Active negotiation thread

Mid-term (deal history)	SQLChatMessageHistory	Past deals for this company
Long-term (preferences)	VectorStore retrieval	Human approval patterns
Shared (feed state)	Supabase + polling	Cross-agent visibility

05 Agent Communication Protocol

The Agent Feed as a LangGraph message channel

Rather than direct agent-to-agent RPC calls, EcoNexus uses an **asynchronous shared feed** — a Supabase table that agents read/write through LangChain tools. This decouples agent lifecycles, enables human observability, and creates a natural audit log.

Post Schema & Routing Logic

agent_post_schema.py

```
from pydantic import BaseModel
from enum import Enum

class PostType(str, Enum):
    OFFER = 'OFFER'
    REQUEST = 'REQUEST'
    NEGOTIATION = 'NEGOTIATION'
    ANNOUNCEMENT = 'ANNOUNCEMENT' # Super Agent only
    DEAL_PROPOSAL = 'DEAL_PROPOSAL' # triggers human interrupt

class AgentPost(BaseModel):
    id: str # uuid
    agent_id: str # e.g. 'agent-acme-munich'
    post_type: PostType
    parent_id: Optional[str] # for threaded replies
    locality: str # geo scope
    visibility: str # 'local' | 'regional' | 'public'
    payload: dict # type-specific fields
    created_at: datetime
    expires_at: Optional[datetime]
```

Real-time Delivery with Supabase Realtime

realtime_feed.py

```
# Agents subscribe to locality channel – no polling needed
supabase.channel('locality:munich')
.on('postgres_changes',
    event='INSERT', schema='public', table='agent_feed',
    filter='locality=eq.munich',
    callback=lambda payload: agent.handle_new_post(payload))
.subscribe()

# LangGraph node wraps the callback
async def listen_feed_node(state: LocalAgentState) -> LocalAgentState:
    new_posts = await feed_queue.get_batch(timeout=5)
    opportunities = score_opportunities(new_posts, state['constraints'])
```

```
return {**state, 'opportunities': opportunities}
```

06 Scaling Strategy

From 3 companies to 10,000 — horizontal + hierarchical

Three Scaling Levers

■ Horizontal Scaling

- One LangGraph worker process per local agent (containerized)
- Agents are stateless between activations — state lives in Postgres checkpointer
- Kubernetes HPA scales worker pods based on feed queue depth
- Target: 1,000+ simultaneous local agents on 4 worker nodes

■ Hierarchical Locality Clustering

- Super Agent created automatically when locality reaches 5+ companies
- Super Agent sub-graphs run in dedicated worker pool (heavier compute)
- Cross-locality fan-out bounded by geographic radius constraint
- Prevents N^2 agent-to-agent explosion: $O(n)$ local + $O(1)$ super per locality

■ Async Event-Driven Execution

- Agents sleep between feed events — near-zero idle cost
- LangGraph's async graph execution with asyncio event loop
- Feed events trigger agent wakeup via Supabase Realtime webhook
- LLM calls batched where possible (e.g., classify 20 offers in one prompt)

Scale Reference Table

Companies	Local Agents	Super Agents	Worker Pods	DB Connections	Est. Monthly Cost
1–10	1–10	0–1	1	5	~\$0 (free tier)
11–50	11–50	1–5	2	20	~\$50
51–200	51–200	5–20	4	50	~\$200
201–1000	201–1000	20–100	8	100	~\$800
1000+	1000+	100+	16+	200+	~\$3000+

Costs are estimates for LLM API + Supabase + compute combined.

07 Human-in-the-Loop Control Layer

LangGraph interrupt_before ensures humans always decide

EcoNexus makes an explicit architectural choice: **agents propose, humans approve**. LangGraph's built-in interrupt mechanism is used at every decision boundary that carries financial or legal consequence.

Interrupt Points

Checkpoint	Graph Node	Human Action Required
Deal Proposal	send_proposal_node	Approve / Reject / Edit terms
High-Value Deal	value_check_node	Extra confirmation for deals > €50k
New Counterparty	trust_check_node	First deal with an unknown company
Multi-Party Symbiosis	symbiosis_review_node	Review 3+ party circular loops
Price Anomaly	anomaly_flag_node	Price deviates > 30% from market
Cross-Locality	cross_local_node	Deal outside defined radius

human_decision_api.py

```
# Human resume pattern – Next.js API route calling LangGraph
# POST /api/deals/{proposal_id}/decision

async def handle_human_decision(proposal_id: str, decision: str, edits: dict):
    thread_config = {'configurable': {'thread_id': proposal_id}}

    # Inject human decision into the interrupted graph
    result = await compiled_agent.ainvoke(
        {'human_decision': decision, 'human_edits': edits},
        config=thread_config
    )

    if decision == 'edit':
        # Agent re-negotiates with human-adjusted terms
        # Graph loops back to negotiate_node automatically
        pass

    return result['active_deals']
```

Agent Configuration UI → Constraint Injection

Human-defined constraints are stored in Supabase and injected into agent state at graph initialization. Changing a constraint takes effect on the next agent activation cycle — no restart needed.

Constraint	Type	Default	Effect on Agent
price_min / price_max	float, per material	Market ± 20%	Hard filter in scan_node
max_distance_km	integer	50 km	Geo filter in scan_node

min_volume_tons	float	1.0	Volume filter
material_whitelist	List[str]	All materials	Restricts OFFER posts
auto_approve_threshold	float	None	Skip interrupt if deal < X
notification_mode	enum	'instant'	How human is alerted

08 Implementation Roadmap

Phased delivery — hackathon to production

Phase 1

Agent Foundation

Week 1–2

- 1 Install LangGraph + LangChain, configure Anthropic API key
- 2 Define all TypedDict state schemas (MFI, Local, Super, Orchestrator)
- 3 Build MFI Agent sub-graph with invoice extraction nodes
- 4 Implement agent_feed Supabase table + post_to_feed_tool
- 5 Basic local agent: scan → negotiate (rule-based) → package → interrupt

Phase 2

Negotiation & Super Agent

Week 3–4

- 1 LLM-powered negotiate_node with NegotiationMove structured output
- 2 Cyclic negotiation loop with round limits and timeout edges
- 3 Super Agent symbiosis detection using networkx cycle detection
- 4 Multi-party deal structuring sub-graph
- 5 PostgresSaver checkpoint for durable state

Phase 3

Specialist Agents & Scaling

Week 5–6

- 1 Recycler, Logistics, Processor, Energy Recovery agents
- 2 Semantic search on agent feed via SupabaseVectorStore
- 3 Orchestrator fan-out using LangGraph Send API
- 4 Kubernetes deployment manifests, HPA config
- 5 Async event-driven activation via Supabase Realtime

Phase 4

Human Control & Production

Week 7–8

- 1 Full interrupt_before / interrupt_after control layer
- 2 Human approval dashboard (Next.js) consuming LangGraph state
- 3 Agent learning: preference extraction from approved/rejected deals
- 4 Digital Passport Agent with QR generation
- 5 Load testing: 100 simultaneous agents, end-to-end negotiation

■ Key Dependencies

langgraph>=0.2 · **langchain**>=0.3 · **langchain-anthropic** · **langchain-community** · **supabase-py** · **networkx** · **pydantic**>=2 · **asyncpg** (PostgresSaver) · **redis** (optional pub/sub)

EcoNexus Agent Architecture

LangChain / LangGraph Technical Reference v1.0

6 Agent Types	Each a dedicated LangGraph StateGraph
Cyclic Negotiation	Multi-round loops with timeout + accept edges
Human Checkpoints	interrupt_before at every financial decision
Zero-Poll Feed	Supabase Realtime events wake sleeping agents
Horizontal Scale	One container per agent, Kubernetes HPA
Durable State	PostgresSaver checkpoints every state transition